

In [1]: `import os`

`print(os.listdir())`

Out:

```
['.ipynb_checkpoints', 'tmdb_5000_credits.csv', 'tmdb_5000_credits.csv.zip', 'Untitled.ipynb', 'Untitled1.ipynb']
```

In [2]: `import pandas as pd`

`credits = pd.read_csv("tmdb_5000_credits.csv")`

`print(credits.head())`

`print("\nColumns:")`

`print(credits.columns)`

Out:

```

      movie_id      title \
0      19995      Avatar
1       285  Pirates of the Caribbean: At World's End
2    206647      Spectre
3     49026  The Dark Knight Rises
4     49529    John Carter

```

```

                                cast \
0  [{"cast_id": 242, "character": "Jake Sully", "...
1  [{"cast_id": 4, "character": "Captain Jack Spa...
2  [{"cast_id": 1, "character": "James Bond", "cr...
3  [{"cast_id": 2, "character": "Bruce Wayne / Ba...
4  [{"cast_id": 5, "character": "John Carter", "c...

```

```

                                crew
0  [{"credit_id": "52fe48009251416c750aca23", "de...
1  [{"credit_id": "52fe4232c3a36847f800b579", "de...
2  [{"credit_id": "54805967c3a36829b5002c41", "de...
3  [{"credit_id": "52fe4781c3a36847f81398c3", "de...
4  [{"credit_id": "52fe479ac3a36847f813eaa3", "de...

```

```
Columns:  
Index(['movie_id', 'title', 'cast', 'crew'], dtype='object')
```

```
In [3]: import os  
        print(os.listdir())
```

```
Out:  
['.ipynb_checkpoints', 'tmdb_5000_credits.csv', 'tmdb_5000_credits.csv.zip', 'Untitled.ipynb', 'Untitled1.ipynb']
```

```
In [4]: import os  
        print(os.listdir())
```

```
Out:  
['.ipynb_checkpoints', 'ml-latest-small.zip', 'tmdb_5000_credits.csv', 'tmdb_5000_credits.csv.zip', 'Untitled.ipynb', 'Untitled1.ipynb']
```

```
In [5]: import zipfile  
  
        with zipfile.ZipFile('ml-latest-small.zip', 'r') as zip_ref:  
            zip_ref.extractall()  
  
        print("Extraction Complete!")
```

```
Out:  
Extraction Complete!
```

```
In [6]: import os  
  
        print(os.listdir())
```

```
Out:  
['.ipynb_checkpoints', 'ml-latest-small', 'ml-latest-small.zip', 'tmdb_5000_credits.csv', 'tmdb_5000_credits.csv.zip', 'Untitled.ipynb', 'Untitled1.ipynb']
```

In [7]: `import os`

```
print(os.listdir('ml-latest-small'))
```

Out:

```
['links.csv', 'movies.csv', 'ratings.csv', 'README.txt', 'tags.csv']
```

In [8]: `import pandas as pd`

```
movies = pd.read_csv('ml-latest-small/movies.csv')
ratings = pd.read_csv('ml-latest-small/ratings.csv')

movies.head()
```

Out:

	movieId	title	genres
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama Romance
4	5	Father of the Bride Part II (1995)	Comedy

In [9]: `print(movies.columns)`
`print(movies.shape)`

Out:

```
Index(['movieId', 'title', 'genres'], dtype='object')
(9742, 3)
```

In [10]: `movies.head()`

Out:

	movieId	title	genres
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama Romance
4	5	Father of the Bride Part II (1995)	Comedy

```
In [11]: movies.info()
```

Out:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9742 entries, 0 to 9741
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   movieId     9742 non-null   int64
1   title       9742 non-null   object
2   genres      9742 non-null   object
dtypes: int64(1), object(2)
memory usage: 228.5+ KB
```

```
In [12]: movies.columns
```

Out:

```
Index(['movieId', 'title', 'genres'], dtype='object')
```

```
In [13]: movies['genres'] = movies['genres'].str.replace('|', ' ', regex=False)

movies.head()
```

Out:

	movieId	title	genres
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy

	movieid	title	genres
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama Romance
4	5	Father of the Bride Part II (1995)	Comedy

```
In [14]: from sklearn.feature_extraction.text import CountVectorizer
        from sklearn.metrics.pairwise import cosine_similarity
```

```
In [15]: cv = CountVectorizer(stop_words='english')

        count_matrix = cv.fit_transform(movies['genres'])

        print(count_matrix.shape)
```

```
Out:
(9742, 23)
```

```
In [16]: indices = pd.Series(
            movies.index,
            index=movies['title']
        ).drop_duplicates()

        indices.head()
```

```
Out:
title
Toy Story (1995)      0
Jumanji (1995)       1
Grumpier Old Men (1995)  2
Waiting to Exhale (1995)  3
Father of the Bride Part II (1995)  4
dtype: int64
```

```

In [17]: def recommend_movie(title):

    if title not in indices:
        return "Movie not found"

    idx = indices[title]

    similarity_scores = list(enumerate(cosine_sim[idx]))

    similarity_scores = sorted(
        similarity_scores,
        key=lambda x: x[1],
        reverse=True
    )

    similarity_scores = similarity_scores[1:6]

    movie_indices = [i[0] for i in similarity_scores]

    return movies['title'].iloc[movie_indices]

```

```

In [18]: recommend_movie('Toy Story (1995)')

```

Out:

```

-----
NameError                                Traceback (most recent call last)
Cell In[18], line 1
----> 1 recommend_movie('Toy Story (1995)')

Cell In[17], line 8, in recommend_movie(title)
      4     return "Movie not found"
      6     idx = indices[title]
----> 8 similarity_scores = list(enumerate(cosine_sim[idx]))
     10 similarity_scores = sorted(
     11     similarity_scores,
     12     key=lambda x: x[1],
     13     reverse=True
     14 )
     16 similarity_scores = similarity_scores[1:6]

NameError: name 'cosine_sim' is not defined

```

```
In [19]: recommend_movie('Jumanji (1995)')
```

Out:

```
-----
NameError                                Traceback (most recent call last)
Cell In[19], line 1
----> 1 recommend_movie('Jumanji (1995)')

Cell In[17], line 8, in recommend_movie(title)
      4     return "Movie not found"
      6     idx = indices[title]
----> 8 similarity_scores = list(enumerate(cosine_sim[idx]))
     10 similarity_scores = sorted(
     11     similarity_scores,
     12     key=lambda x: x[1],
     13     reverse=True
     14 )
     16 similarity_scores = similarity_scores[1:6]

NameError: name 'cosine_sim' is not defined
```

```
In [20]: import matplotlib.pyplot as plt

genre_series = movies['genres'].str.split()

genre_list = []

for row in genre_series:
    genre_list.extend(row)

genre_count = pd.Series(genre_list).value_counts()

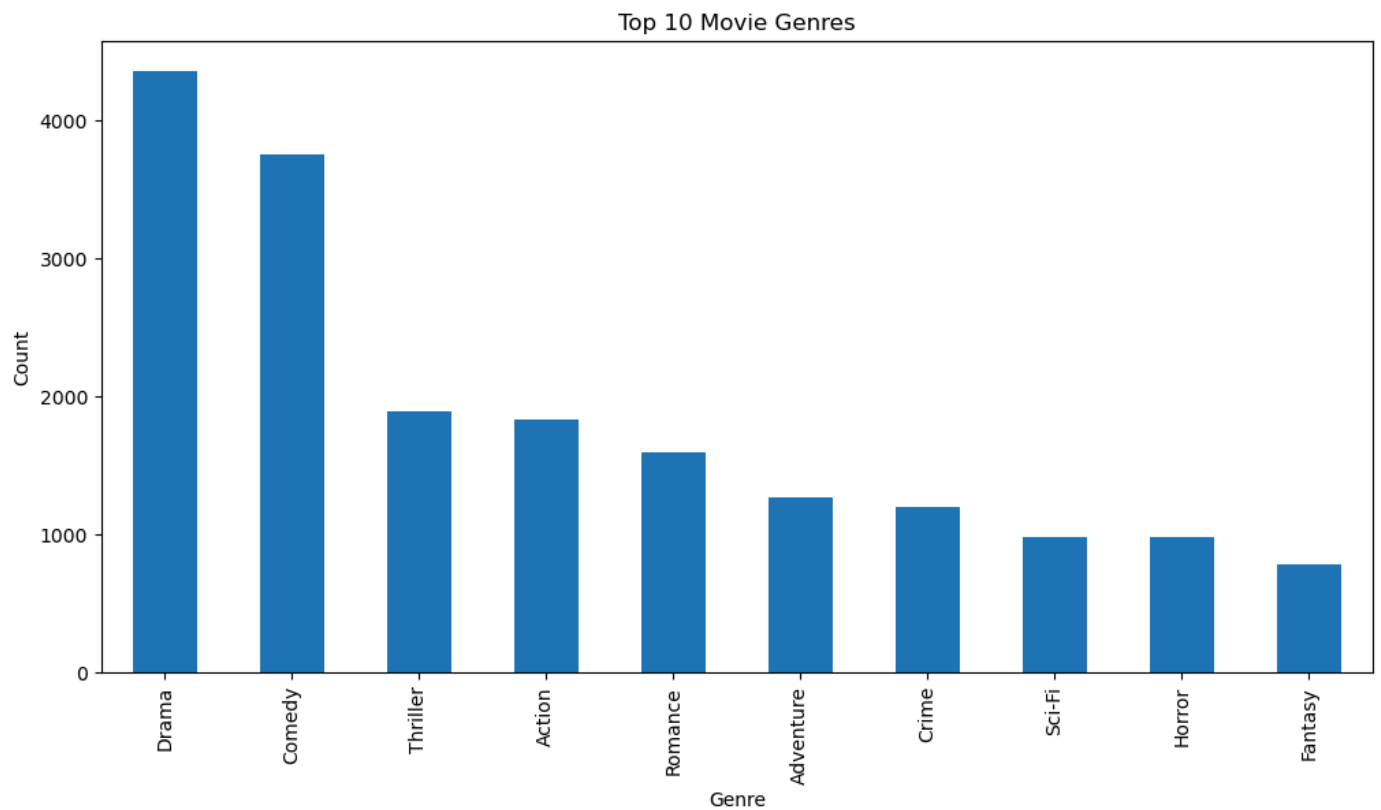
plt.figure(figsize=(12,6))

genre_count.head(10).plot(kind='bar')

plt.title('Top 10 Movie Genres')
plt.xlabel('Genre')
plt.ylabel('Count')

plt.show()
```

Out:



```
In [21]: print("Total Movies:", len(movies))

print("Most Common Genre:", genre_count.idxmax())

print("Number of Genres:", len(genre_count))
```

Out:

```
Total Movies: 9742
Most Common Genre: Drama
Number of Genres: 22
```